

Backtracking y SLD

Informe de la actividad grupal - Programación III (Grupo 1)

Prolog: grafo dirigido de ciudades de Canadá, reglas y consultas

Integrantes	Nicolás Tovar Gaviria - Santiago Velásquez
Profesor	Ramiro A. Barrios Valencia
Entrega	6 de octubre de 2026
Repositorio / Drive	[Enlace al repositorio de GitHub o carpeta de Drive]

1. Descripción del problema

La actividad parte del grafo dirigido de la diapositiva "Y si se filtra? grafos dirigidos", en el que cada arista tiene un sentido y un costo. El objetivo es representarlo en Prolog, agregar las reglas que pide el profesor y responder las consultas propuestas, observando cómo Prolog resuelve cada una mediante resolución SLD y backtracking.

Las consultas y reglas pedidas son: (a) ¿existe conexión entre Saskatoon y Vancouver?; (b) ¿con qué nodos está conectado Regina y cuál es el costo de cada conexión?; (c) una regla para saber si un nodo tiene aristas; (d) una regla para calcular el costo de ir de un nodo X a un nodo Z pasando por Y; y (e) la pregunta más interesante: ¿es posible viajar desde Edmonton a Calgary?

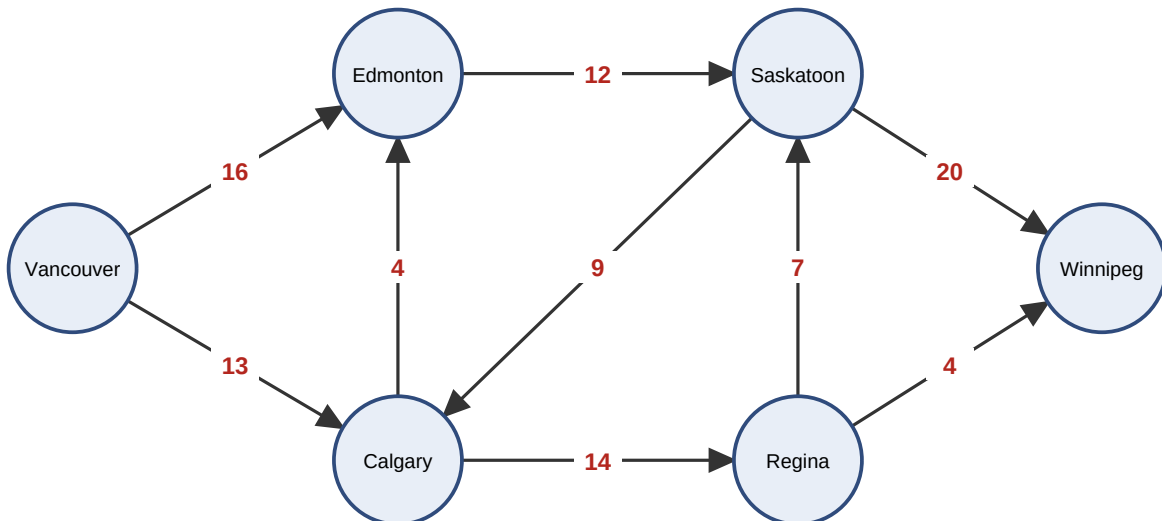


Figura 1. Grafo dirigido (los números son los costos de cada arista).

2. Programa en Prolog

2.1 Base de hechos

Cada arista se representa con el hecho `arista(Origen, Destino, Costo)`. La correspondencia con los nodos de la diapositiva es la siguiente:

Origen	Destino	Costo	En la diapositiva
vancouver	edmonton	16	s -> v1
vancouver	calgary	13	s -> v2
calgary	edmonton	4	v2 -> v1
edmonton	saskatoon	12	v1 -> v3
saskatoon	calgary	9	v3 -> v2
calgary	regina	14	v2 -> v4
regina	saskatoon	7	v4 -> v3
saskatoon	winnipeg	20	v3 -> t
regina	winnipeg	4	v4 -> t

2.2 Reglas y código completo (archivo grafo.pl)

El grafo tiene un ciclo (calgary -> regina -> saskatoon -> calgary). Por eso la regla camino/4 lleva una lista de nodos visitados: sin ella, el backtracking entraría en un recorrido infinito. Esto hace que los caminos sean simples (ningún nodo se repite).

```
% =====
% Actividad: Backtracking y SLD
% Grafo dirigido de ciudades de Canada (diapositiva "Y si se filtra?")
% arista(Origen, Destino, Costo).
% =====

% --- Hechos: aristas dirigidas con su costo ---
arista(vancouver, edmonton, 16). % s -> v1
arista(vancouver, calgary, 13). % s -> v2
arista(calgary, edmonton, 4). % v2 -> v1
arista(edmonton, saskatoon, 12). % v1 -> v3
arista(saskatoon, calgary, 9). % v3 -> v2
arista(calgary, regina, 14). % v2 -> v4
arista(regina, saskatoon, 7). % v4 -> v3
arista(saskatoon, winnipeg, 20). % v3 -> t
arista(regina, winnipeg, 4). % v4 -> t

% --- Regla 1: camino entre dos nodos ---
% camino(X, Z, Costo, Ruta): hay un camino dirigido de X a Z con ese
% costo total. Se lleva la lista de nodos visitados para evitar ciclos
% (el grafo tiene el ciclo calgary -> regina -> saskatoon -> calgary,
% sin esa lista el backtracking no terminaria). Los caminos son simples:
% ningun nodo se repite, por lo que conectado(X, X) es siempre falso.
camino(X, Z, Costo, Ruta) :-
    cam(X, Z, [X], Costo, RutaInv),
    reverse(RutaInv, Ruta).

cam(X, Z, Visitados, Costo, [Z|Visitados]) :-
    arista(X, Z, Costo),
    \+ member(Z, Visitados).
cam(X, Z, Visitados, Costo, Ruta) :-
    arista(X, W, C1),
    \+ member(W, Visitados),
    cam(W, Z, [W|Visitados], C2, Ruta),
    Costo is C1 + C2.

% --- Regla 2: conexion (existe algun camino) ---
```

```

conectado(X, Z) :-
    camino(X, Z, _, _).

% --- Regla 3: un nodo tiene aristas (entrantes o salientes) ---
tiene_aristas(N) :-
    arista(N, _, _).
tiene_aristas(N) :-
    arista(_, N, _).

% --- Regla 4: costo de ir de X a Z pasando por Y ---
% Se busca un camino X -> Z cuya ruta contenga a Y como nodo intermedio.
costo_por(X, Y, Z, Costo) :-
    camino(X, Z, Costo, Ruta),
    member(Y, Ruta),
    Y \== X,
    Y \== Z.

% --- Regla 5 (apoyo): conexiones de un nodo con su costo ---
conexion(X, Z, Costo) :-
    camino(X, Z, Costo, _).

```

2.3 Resumen de las reglas

Regla	Qué hace
camino(X, Z, Costo, Ruta)	Hay un camino dirigido de X a Z; devuelve el costo total y la lista de nodos recorridos.
conectado(X, Z)	Es verdadera si existe algún camino de X a Z.
conexion(X, Z, Costo)	Lista cada destino alcanzable desde X con el costo de cada ruta.
tiene_aristas(N)	N es origen o destino de alguna arista.
costo_por(X, Y, Z, Costo)	Costo de un camino de X a Z que pasa por Y como nodo intermedio.

3. Backtracking y resolución SLD en la consulta interesante

Se traza ?- conectado(edmonton, calgary) ., que Prolog resuelve así (la meta se reduce con la regla de conectado a camino y de ahí a cam/5 con Visitados = [edmonton]):

Paso	Meta que Prolog intenta	Resultado
1	cam(edmonton, calgary, [edmonton], ...) - cláusula 1: arista(edmonton, calgary, C)	Falla: no existe ese hecho. Backtracking a la cláusula 2.
2	Cláusula 2: arista(edmonton, W, C1)	W = saskatoon, C1 = 12 (único hecho con origen edmonton). saskatoon no está visitado.
3	cam(saskatoon, calgary, [saskatoon, edmonton], ...) - cláusula 1: arista(saskatoon, calgary, 9)	Éxito: C2 = 9, calgary no está visitado.
4	Costo is 12 + 9	Costo = 21. Ruta = [edmonton, saskatoon, calgary]. La consulta responde true.
5	Backtracking en busca de más soluciones: saskatoon -> winnipeg, y saskatoon -> calgary -> regina -> ...	Todas las ramas fallan: terminan en nodos ya visitados o en winnipeg, que no tiene aristas salientes. No hay más soluciones.

4. Consultas y resultados

Los resultados siguientes son la salida real del script `consultas.pl`, ejecutado con SWI-Prolog (`swipl -q -g ejecutar -t halt consultas.pl`).

4.1 Preguntas de la diapositiva

```
?- conectado(saskatoon, vancouver).
false.

?- conectado(vancouver, saskatoon).
true.

?- conexion(regina, Z, C). % formato Z-C
saskatoon-7
winnipeg-4
calgary-16
winnipeg-27
edmonton-20

?- camino(regina, Z, C, Ruta). % formato Z-C-Ruta
saskatoon-7-[regina,saskatoon]
winnipeg-4-[regina,winnipeg]
calgary-16-[regina,saskatoon,calgary]
winnipeg-27-[regina,saskatoon,winnipeg]
edmonton-20-[regina,saskatoon,calgary,edmonton]

?- costo_minimo_de_regina_a_cada_destino
calgary: 16
edmonton: 20
saskatoon: 7
winnipeg: 4
```

¿Existe conexión entre Saskatoon y Vancouver? No (false). Ninguna arista llega a Vancouver: solo tiene aristas salientes. La conexión sí existe en el sentido contrario: `conectado(vancouver, saskatoon)` es true. Por eso el sentido de las aristas importa en un grafo dirigido.

¿Con qué nodos está conectado Regina y a qué costo? Regina alcanza cuatro nodos. Winnipeg tiene dos rutas distintas, y para cada destino se incluye el menor costo:

Destino	Rutas encontradas (costo)	Costo mínimo
saskatoon	regina, saskatoon (7)	7
winnipeg	regina, winnipeg (4); regina, saskatoon, winnipeg (27)	4
calgary	regina, saskatoon, calgary (16)	16
edmonton	regina, saskatoon, calgary, edmonton (20)	20

4.2 Reglas nuevas

```
?- tiene_aristas(vancouver).
true.

?- tiene_aristas(winnipeg).
true.

?- tiene_aristas(toronto).
false.
```

```

?- setof(N, tiene_aristas(N), L).
[calgary,edmonton,regina,saskatoon,vancouver,winnipeg]

?- costo_por(vancouver, saskatoon, winnipeg, C). % formato C
48
55
49
54

?- costo_por(vancouver, calgary, regina, C). % formato C
51
27

?- costo_por(vancouver, regina, edmonton, C). % formato C
false.

?- costo_por(edmonton, Y, winnipeg, C). % formato Y-C
saskatoon-32
saskatoon-39
calgary-39
regina-39

```

tiene_aristas/1. Es true para vancouver y winnipeg (este último solo recibe aristas) y false para toronto, que no aparece en el grafo. La consulta con `setof` lista los seis nodos del grafo.

costo_por/4. De Vancouver a Winnipeg pasando por Saskatoon hay cuatro rutas, con costos 48, 49, 54 y 55 (la más barata es vancouver, edmonton, saskatoon, winnipeg). De Vancouver a Regina pasando por Calgary hay dos: 27 (directa por Calgary) y 51 (vancouver, edmonton, saskatoon, calgary, regina). De Vancouver a Edmonton pasando por Regina no hay ninguna: los únicos caminos simples son vancouver-edmonton y vancouver-calgary-edmonton, y ninguno pasa por Regina (hacerlo exigiría repetir Calgary). Con la variable `Y` sin ligar, `costo_por(edmonton, Y, winnipeg, C)` enumera los intermedios posibles: saskatoon (32 y 39), y calgary y regina (39).

4.3 Pregunta más interesante: ¿es posible viajar de Edmonton a Calgary?

```

?- conectado(edmonton, calgary).
true.

?- camino(edmonton, calgary, C, Ruta). % formato C-Ruta
21-[edmonton,saskatoon,calgary]

?- conectado(calgary, edmonton).
true.

?- camino(calgary, edmonton, C, Ruta). % formato C-Ruta
4-[calgary,edmonton]

```

Sí es posible, aunque no hay una arista directa de Edmonton a Calgary. El único camino es edmonton, saskatoon, calgary, con costo $12 + 9 = 21$. En cambio, de Calgary a Edmonton sí existe una arista directa de costo 4: la existencia de la conexión depende del sentido de las aristas.

5. Observaciones

Los caminos son simples, así que `conectado(X, X)` siempre es false aunque exista un ciclo que vuelva al nodo de partida (por ejemplo, regina, saskatoon, calgary, regina). El orden de las respuestas depende del orden de los hechos en la base, que es el orden en que el backtracking los prueba.

6. Anexo: script de consultas (consultas.pl)

```
:- consult('grafo.pl').

% si_no(+Meta): imprime true/false para consultas de si/no
si_no(Consulta, Meta) :-
    format("~n?- ~w~n", [Consulta]),
    (   call(Meta) -> writeln('true.') ; writeln('false.') ).

% todas(+Consulta, +Plantilla, :Meta): imprime cada solucion de Meta
todas(Consulta, Plantilla, Meta) :-
    format("~n?- ~w~n", [Consulta]),
    findall(Plantilla, Meta, Lista),
    (   Lista == []
    -> writeln('false.')
    ;   forall(member(L, Lista), (print(L), nl))
    ).

ejecutar :-
    writeln('##### PREGUNTAS DE LA DIAPOSITIVA #####'),

    % 1. Existe conexion entre Saskatoon y Vancouver?
    si_no('conectado(saskatoon, vancouver).', conectado(saskatoon, vancouver)),
    si_no('conectado(vancouver, saskatoon).', conectado(vancouver, saskatoon)),

    % 2. Con que nodos esta conectado Regina y costo de cada conexion?
    todas('conexion(regina, Z, C). % formato Z-C',
          Z-C, conexion(regina, Z, C)),
    todas('camino(regina, Z, C, Ruta). % formato Z-C-Ruta',
          Z-C-R, camino(regina, Z, C, R)),

    % Costo minimo por destino
    format("~n?- costo minimo de regina a cada destino~n"),
    setof(Z2, C2^conexion(regina, Z2, C2), Destinos),
    forall(member(D, Destinos),
           ( aggregate_all(min(Cm), conexion(regina, D, Cm), Min),
             format("~w: ~w~n", [D, Min]) )),

    writeln('\n##### REGLAS NUEVAS #####'),

    % 3. Regla: un nodo tiene aristas
    si_no('tiene_aristas(vancouver).', tiene_aristas(vancouver)),
    si_no('tiene_aristas(winnipeg).', tiene_aristas(winnipeg)),
    si_no('tiene_aristas(toronto).', tiene_aristas(toronto)),
    format("~n?- setof(N, tiene_aristas(N), L).~n"),
    setof(N, tiene_aristas(N), Nodos), print(Nodos), nl,

    % 4. Regla: costo de X a Z pasando por Y
    todas('costo_por(vancouver, saskatoon, winnipeg, C). % formato C',
          C4, costo_por(vancouver, saskatoon, winnipeg, C4)),
    todas('costo_por(vancouver, calgary, regina, C). % formato C',
          C5, costo_por(vancouver, calgary, regina, C5)),
    todas('costo_por(vancouver, regina, edmonton, C). % formato C',
          C6, costo_por(vancouver, regina, edmonton, C6)),
    todas('costo_por(edmonton, Y, winnipeg, C). % formato Y-C',
          Y-C7, costo_por(edmonton, Y, winnipeg, C7)),

    writeln('\n##### PREGUNTA MAS INTERESANTE #####'),

    % 5. Es posible viajar de Edmonton a Calgary?
    si_no('conectado(edmonton, calgary).', conectado(edmonton, calgary)),
    todas('camino(edmonton, calgary, C, Ruta). % formato C-Ruta',
          C8-R8, camino(edmonton, calgary, C8, R8)),
    si_no('conectado(calgary, edmonton).', conectado(calgary, edmonton)),
```

```
todas('camino(calgary, edmonton, C, Ruta). % formato C-Ruta',  
      C9-R9, camino(calgary, edmonton, C9, R9)).
```

7. Participación del grupo y soportes

Completar antes de entregar. Cada integrante debe entregar el mismo PDF y el mismo enlace con el versionado.

Integrante	Aporte
Nicolás Tovar Gaviria	[Describir aporte]
Santiago Velásquez	[Describir aporte]

Enlace a los soportes (GitHub o Drive con versionado): [pegar enlace aquí]